

Übungsblatt 2: Scheduling-Theorie und kooperativer Scheduler

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Übung verknüpfen Sie die mathematische Scheduling-Theorie mit der praktischen Software-Entwicklung. Zunächst analysieren Sie ein Task-Set auf seine Planbarkeit mittels Rate Monotonic Scheduling (RMS). Anschließend implementieren Sie auf dem STM32H755 (Cortex-M4) Ihren eigenen kleinen kooperativen Round-Robin-Scheduler in C, um das Konzept des Task Control Blocks (TCB) und des Kontextwechsels zu verstehen. Abschließend messen Sie die Schwächen dieses Systems am Oszilloskop und provozieren einen System-Crash.

Aufgabe 1: Scheduling-Theorie (RMS-Schranke)

Ein Steuergerät soll drei periodische Tasks verarbeiten. Für die Tasks wurden folgende Worst-Case Execution Times (C_i) und Perioden (T_i) ermittelt (Implicit Deadlines, $D_i = T_i$):

- τ_1 : $C_1 = 1$ ms, $T_1 = 4$ ms
- τ_2 : $C_2 = 2$ ms, $T_2 = 6$ ms
- τ_3 : $C_3 = 1$ ms, $T_3 = 12$ ms

1. Berechnen Sie die Gesamt-CPU-Auslastung U .
2. Vergeben Sie die Prioritäten der Tasks nach den Regeln des Rate Monotonic Scheduling (RMS).
3. Prüfen Sie mit der Liu-und-Layland-Schranke $U \leq n(2^{1/n} - 1)$, ob dieses System garantiert (hinreichend) schedulbar ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

1. Berechnung der Auslastung U :

$$U = \sum_{i=1}^3 \frac{C_i}{T_i} = \frac{1}{4} + \frac{2}{6} + \frac{1}{12} = \frac{3}{12} + \frac{4}{12} + \frac{1}{12} = \frac{8}{12} = \frac{2}{3} \approx 0,667 \text{ (66,7\%)}$$

2. Prioritäten nach RMS:

Regel: Je kürzer die Periode T_i , desto höher die Priorität.

- Höchste Priorität: τ_1 ($T_1 = 4$ ms)
- Mittlere Priorität: τ_2 ($T_2 = 6$ ms)
- Niedrigste Priorität: τ_3 ($T_3 = 12$ ms)

3. Test der Liu-und-Layland-Schranke:

Bei $n = 3$ Tasks gilt $n(2^{1/n} - 1) = 3(2^{1/3} - 1) \approx 0,780$ (78,0%).

Da $U = \frac{2}{3} \approx 66,7\%$ kleiner ist als diese Schranke, liegt das System sicher in Zone 1 und ist **garantiert schedulbar** (hinreichendes Kriterium erfüllt).

Aufgabe 2: Die TCB-Datenstruktur anlegen

Wir bauen nun in der CubeIDE (Projekt auf dem Cortex-M4) einen simplen C-Scheduler. Definieren Sie in Ihrer `main.c` (im Bereich `USER CODE`) einen Task Control Block (TCB). Da wir noch keinen echten Assembler-Registertausch durchführen, benötigen wir keinen Stack-Pointer. Wir nutzen einen Funktionszeiger.

1. Erstellen Sie den `typedef` für einen Funktionszeiger `TaskFunction_t`.
2. Erstellen Sie das `struct MyTCB_t` mit den Variablen für den Funktionszeiger, den Zustand (`state`) und einer Schlaf-Zeit (`sleep_time`).
3. Legen Sie ein globales Array `task_list[3]` vom Typ `MyTCB_t` und einen Index `current_task_idx` an.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

```
/* USER CODE BEGIN 0 */
#define READY 1
#define BLOCKED 0

typedef void (*TaskFunction_t)(void);

typedef struct {
    TaskFunction_t task_func; /* welcher C-Code? */
    uint8_t state;           /* READY oder BLOCKED */
    uint32_t sleep_time;     /* für os_delay() */
} MyTCB_t;

MyTCB_t task_list[3];
uint8_t current_task_idx = 0;
/* USER CODE END 0 */
```

Aufgabe 3: Die Scheduler-Logik (os_yield)

Implementieren Sie die Funktion `os_yield()`. Diese Funktion wird später von den Tasks freiwillig aufgerufen, um die CPU abzugeben (kooperatives Scheduling). Die Funktion soll ringförmig (Round-Robin) zum nächsten Index springen (modulo 3), prüfen, ob der Zustand der dort liegenden Task `READY` ist, und anschließend den Funktionszeiger der neuen Task ausführen.

Hinweis (Didaktik): Der direkte Aufruf von `task_func()` aus `os_yield()` ist eine bewusste Vereinfachung. Bei `while(1) + Yield` wächst der Call-Stack (kein echter Context Switch). In einem Produktions-OS kehrt man zum Dispatcher zurück.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

```
/* USER CODE BEGIN 4 */
void os_yield(void)
{
    do {
        /* Index ringförmig hochzählen */
        current_task_idx = (current_task_idx + 1) % 3;
    } while (task_list[current_task_idx].state != READY);

    /* Nächste bereite Task ausführen (didaktisch: Stack wächst!) */
    task_list[current_task_idx].task_func();
}
/* USER CODE END 4 */
```

Hinweis: In Produktion würde `os_yield` nicht die Task-Funktion aufrufen, sondern zum Dispatcher zurückkehren — sonst wächst der Stack mit jedem Kontextwechsel.

Aufgabe 4: Tasks ausführen & Jitter messen (Praxis)

Schreiben Sie drei kurze Funktionen `Task1()`, `Task2()` und `Task3()`.

1. Lassen Sie in `Task1()` einen GPIO-Pin auf `HIGH` und direkt vor dem Aufruf von `os_yield()` wieder auf `LOW` wechseln, um einen Messpunkt zu generieren. Die anderen Tasks können leere Schleifen-Durchläufe oder LED-Toggles beinhalten. Am Ende jeder Task muss `os_yield()` aufgerufen werden!
2. Füllen Sie das `task_list`-Array in der `main()` mit Ihren Funktionen, setzen Sie den Status auf `READY` und rufen Sie den ersten Funktionszeiger manuell auf, um das System zu starten.
3. Messen Sie den GPIO-Pin von `Task1()` am Oszilloskop. Was fällt Ihnen beim Timing auf?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Beispielhafter Code für Tasks und Initialisierung:

```
void Task1(void)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
    /* ... etwas Arbeit ... */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
    os_yield(); /* CPU freiwillig abgeben */
}

void Task2(void)
{
    /* ... andere Arbeit ... */
    os_yield();
}

void Task3(void)
{
    os_yield();
}

/* In main(): */
task_list[0].task_func = Task1;
task_list[0].state = READY;
task_list[1].task_func = Task2;
task_list[1].state = READY;
task_list[2].task_func = Task3;
task_list[2].state = READY;

/* Startschuss: */
task_list[0].task_func();
```

Erwartetes Messergebnis am Oszilloskop:

Sie werden feststellen, dass der Startzeitpunkt von `Task1()` schwankt. Dieses System hat einen hohen Jitter. Der Aufruf von `Task1()` hängt exakt davon ab, wie viel C-Code `Task2()` und `Task3()` abarbeiten, bevor sie wieder `os_yield()` aufrufen.

Aufgabe 5: Den Scheduler crashen

Wir zeigen nun die größte Schwachstelle von kooperativen Systemen auf.

1. Fügen Sie in `Task2()` eine lange Verzögerung ein (z. B. `HAL_Delay(1000)` oder eine endlose `while(1)`-Schleife). Verändern Sie nicht den Code in `Task1()` oder `Task3()`.

2. Flashen Sie das Programm und beobachten Sie das Oszilloskop und die LEDs.
3. Erklären Sie das beobachtete Verhalten und warum dies in echten Echtzeitsystemen inakzeptabel ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 5

Erklärung des System-Crashes / der Starvation:

Auf dem Oszilloskop ist `Task1()` sofort eingefroren oder stottert massiv, da das Puls-Signal ausbleibt. In einem kooperativen System gibt die laufende Task die CPU nur durch Aufruf von `os_yield()` ab. Wenn `Task2()` in einer Endlosschleife hängt oder ein langes Delay aufruft, ohne `os_yield()` zu feuern, erhält der Scheduler nie wieder die Kontrolle zurück. Alle anderen Tasks *verhungern* (Starvation).

Dieses Verhalten ist für harte Echtzeitsysteme absolut inakzeptabel: Ein Fehler in einer unwichtigen Hintergrund-Task darf niemals eine kritische Task stoppen. Deshalb verwenden moderne Systeme wie FreeRTOS präemptives Scheduling, bei dem ein Hardware-Timer (SysTick) der Task gewaltsam die CPU entzieht.